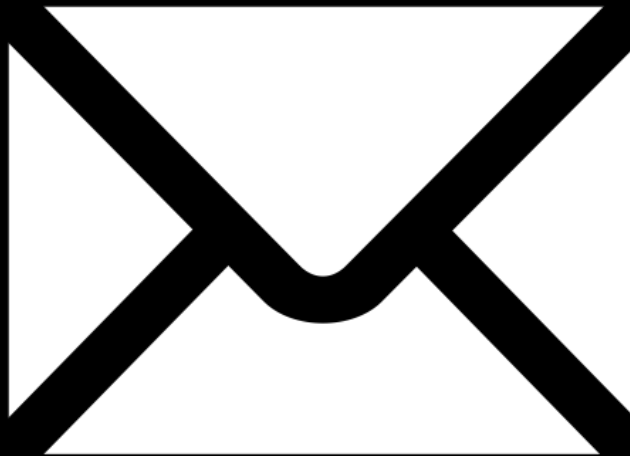


24: Linear Regression and Gradient Ascent

Jerry Cain and Matt Harvill

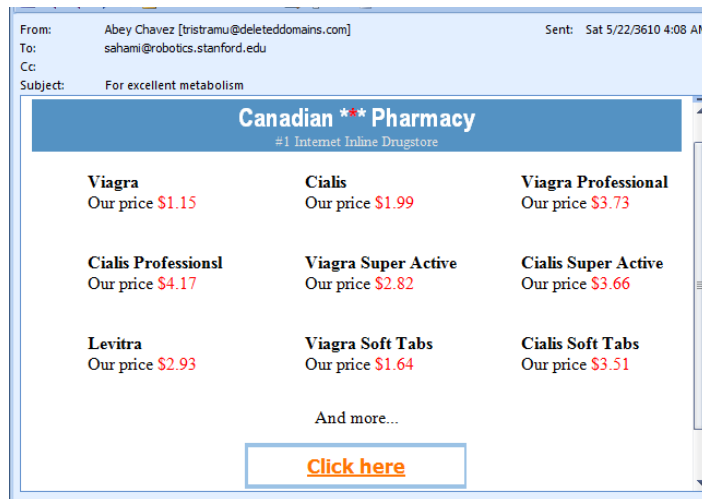
May 24, 2024

[Lecture Discussion on Ed](#)



and Learn
naively

What is Bayes doing in my mail server?



Let's get Bayesian on your spam:

Content analysis details:

0.9 RCVD_IN_PBL

1.5 URIBL_WS_SURBL

5.0 URIBL_JP_SURBL

5.0 URIBL_OB_SURBL

5.0 URIBL_SC_SURBL

2.0 URIBL_BLACK

8.0 BAYES_99

(49.5 hits, 7.0 required)

RBL: Received via a relay in Spamhaus PBL
[93.40.189.29 listed in zen.spamhaus.org]

Contains an URL listed in the WS SURBL blocklist
[URIs: recragas.cn]

Contains an URL listed in the JP SURBL blocklist
[URIs: recragas.cn]

Contains an URL listed in the OB SURBL blocklist
[URIs: recragas.cn]

Contains an URL listed in the SC SURBL blocklist
[URIs: recragas.cn]

Contains an URL listed in the URIBL blacklist
[URIs: recragas.cn]

BODY: Bayesian spam probability is 99 to 100%
[score: 1.0000]

A Bayesian Approach to Filtering Junk E-Mail

Mehran Sahami* Susan Dumais† David Heckerman† Eric Horvitz†

*Gates Building 1A
Computer Science Department
Stanford University
Stanford, CA 94305-9010
sahami@cs.stanford.edu

†Microsoft Research
Redmond, WA 98052-6399
{sdumais, heckerma, horvitz}@microsoft.com

Abstract

In addressing the growing problem of junk E-mail on the Internet, we examine methods for the automated

contain offensive material (such as graphic pornography), there is often a higher cost to users of actually viewing this mail than simply the time to sort out the junk. For the junk mail not only wastes more time, but

Email classification

Goal Based on email content $\mathbf{X}$, predict if email is spam or not.

Features Consider a lexicon m words (for English: $m \approx 100,000$).

$\mathbf{X} = (X_1, X_2, \dots, X_m)$, m indicator variables

$X_j = 1$ if word j appeared in document

Output $Y = 1$ if email is spam

Note: m is huge. Make Naïve Bayes assumption: $P(\mathbf{X}|\text{spam}) = \prod_{j=1}^m P(X_j|\text{spam})$

Appearances of words in email are conditionally independent
given the email is spam or not

Training: Naïve Bayes Email classification

Train set

- n previous emails $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$
- $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_m^{(i)})$ for each word, whether it appears in email i
- $y^{(i)} = 1$ if spam, 0 if not spam

Note: m is huge.

Which estimator should we use for $\hat{P}(X_j|Y)$?

- A. MLE
- B. Laplace estimate (MAP)**
- C. Other MAP estimate
- D. Both A and B

Many words are likely to not appear at all in the training set!



Ex 3. Naïve Bayes Classifier (m, n large)

$$\hat{Y} = \arg \max_{y=\{0,1\}} \left(\prod_{j=1}^m \hat{P}(X_j|Y) \right) \hat{P}(Y)$$

Training

$\forall j$: $\hat{P}(X_j = 1|Y = 0), \hat{P}(X_j = 0|Y = 0),$ Use MLE or
 $\hat{P}(X_j = 1|Y = 1), \hat{P}(X_j = 0|Y = 1),$ Laplace (MAP)
 $\hat{P}(Y = 1), \hat{P}(Y = 0)$

Testing

$$\hat{Y} = \arg \max_{y=\{0,1\}} \left(\prod_{j=1}^m \right)$$

Laplace (MAP) estimates avoid estimating 0 probabilities for events that don't occur in your training data.

Testing: Naïve Bayes Email classification

For a new email:

- Generate $\mathbf{X} = (X_1, X_2, \dots, X_m)$
- Classify as spam or not using Naïve Bayes assumption

Note: m is huge.

Suppose train set size n also huge (many labeled emails).

Can we still use the below prediction?

$$\hat{Y} = \arg \max_{y=\{0,1\}} \left(\prod_{j=1}^m \hat{P}(X_j|Y) \right) \hat{P}(Y)$$

Testing: Naïve Bayes Email classification

For a new email:

- Generate $\mathbf{X} = (X_1, X_2, \dots, X_m)$
- Classify as spam or not using Naïve Bayes assumption

Note: m is huge.

Suppose train set size n also huge (many labeled emails).

Can we still use the below prediction?

$$\hat{Y} = \arg \max_{y=\{0,1\}} \left(\prod_{j=1}^m \hat{P}(X_j|Y) \right) \hat{P}(Y)$$

Will probably lead to underflow!

Ex 3. Naïve Bayes Classifier (m, n large)

$$\hat{Y} = \arg \max_{y=\{0,1\}} \left(\prod_{j=1}^m \hat{P}(X_j|Y) \right) \hat{P}(Y)$$

Training

$$\forall i: \hat{P}(X_j = 1|Y = 0), \hat{P}(X_j = 0|Y = 0), \hat{P}(X_j = 1|Y = 1), \hat{P}(X_j = 0|Y = 1), \hat{P}(Y = 1), \hat{P}(Y = 0)$$

Use sums of log-probabilities for numerical stability.

Testing

$$\hat{Y} = \arg \max_{y=\{0,1\}} \left(\log \hat{P}(Y) + \sum_{j=1}^m \log \hat{P}(X_j|Y) \right)$$

How well does Naïve Bayes perform?

After training, you can test with another set of data, called the **test set**.

- Test set also has known values for Y so we can see how often we were right/wrong in our predictions $\hat{Y}$.

Typical workflow:

- Have a dataset of 1789 emails (1578 spam, 211 ham)
- Train set: First 1538 emails (by time)
- Test set: Next 251 messages

Evaluation criteria on test set:

$$\text{precision} = \frac{(\# \text{ correctly predicted class } Y)}{(\# \text{ predicted class } Y)}$$

$$\text{recall} = \frac{(\# \text{ correctly predicted class } Y)}{(\# \text{ real class } Y \text{ messages})}$$

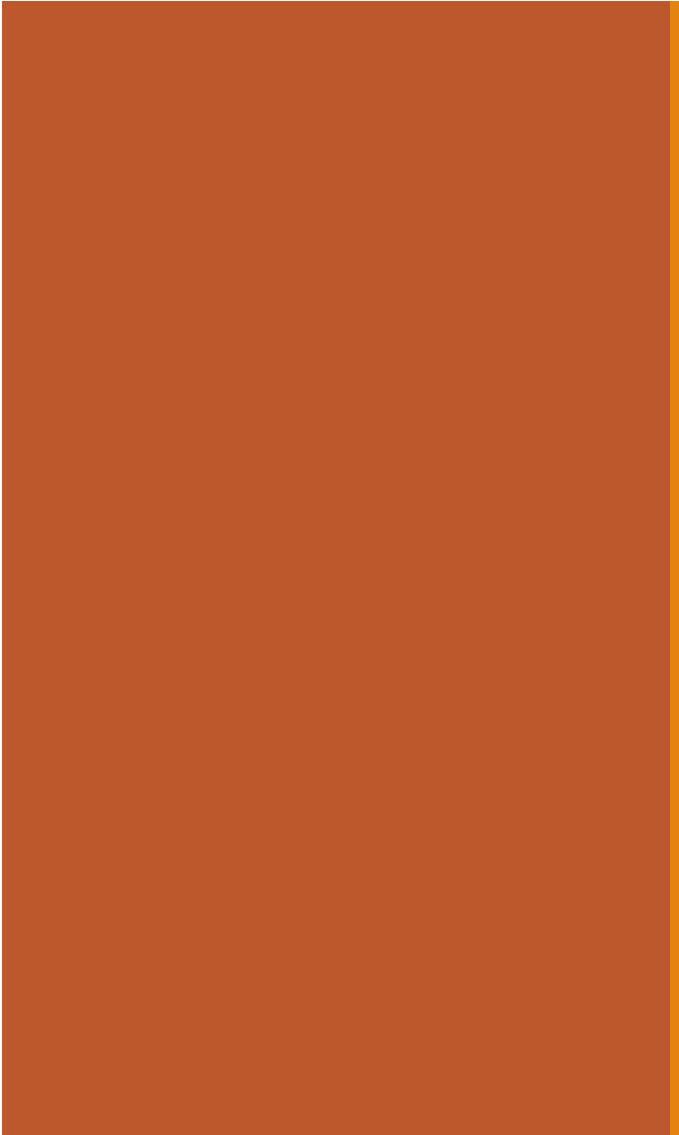
	Spam		Non-spam	
	Prec.	Recall	Prec.	Recall
Words only	97.1%	94.3%	87.7%	93.4%
Words + addtl features	100%	98.3%	96.2%	100%

What are precision and recall?

Accuracy ($\# \text{ correct} / \# \text{ total}$) sometimes just doesn't cut it.

Precision:	Of the emails you predicted as spam, how many are <i>truly</i> spam?	Measure of false positives
Recall:	Of the emails that are truly spam, how many did you predict?	Measure of false negatives

More on Wikipedia (https://en.wikipedia.org/wiki/Precision_and_recall)



Linear Regression

Today's goals

We are going to learn linear regression.

- Informally known as "fitting data to a straight line"
- Linear models, admittedly, are often too simple for complex datasets
- Many tasks in computer science call for classification, not regression



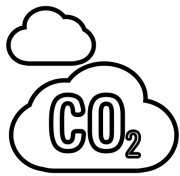
We still cover this topic so we can learn **compelling techniques** that will help us design and understand more complicated learning algorithms:

1. How to model likelihood of training data $(\mathbf{x}^{(i)}, y^{(i)})$
2. What rules of calculus and **argmax** are important to remember
3. What **gradient ascent** is and why it is useful

Regression: Predicting real numbers

Review

Training data: $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$

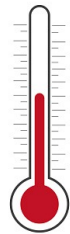


CO2 levels

Year 1	338.8
Year 2	340.0
...	
Year n	340.76

$\mathbf{X} = (X_1)$

(assume one feature)



Global Land-Ocean temperature

Output

0.26
0.32
⋮
0.14

$Y \in \mathbb{R}$

Model:

*this is our predictor
Ŷ for unknown data*

$\hat{Y} = g(\mathbf{X}),$
for some parametric
function g

Linear Regression

Assume linear model
(and $\mathbf{X}$ is 1-D):

$$\hat{Y} = g(\mathbf{X}) = aX + b$$

*these are unknowns,
need to infer
the best choices
for them.*

*$= ax_i + b$ } only one
x input for
this example*

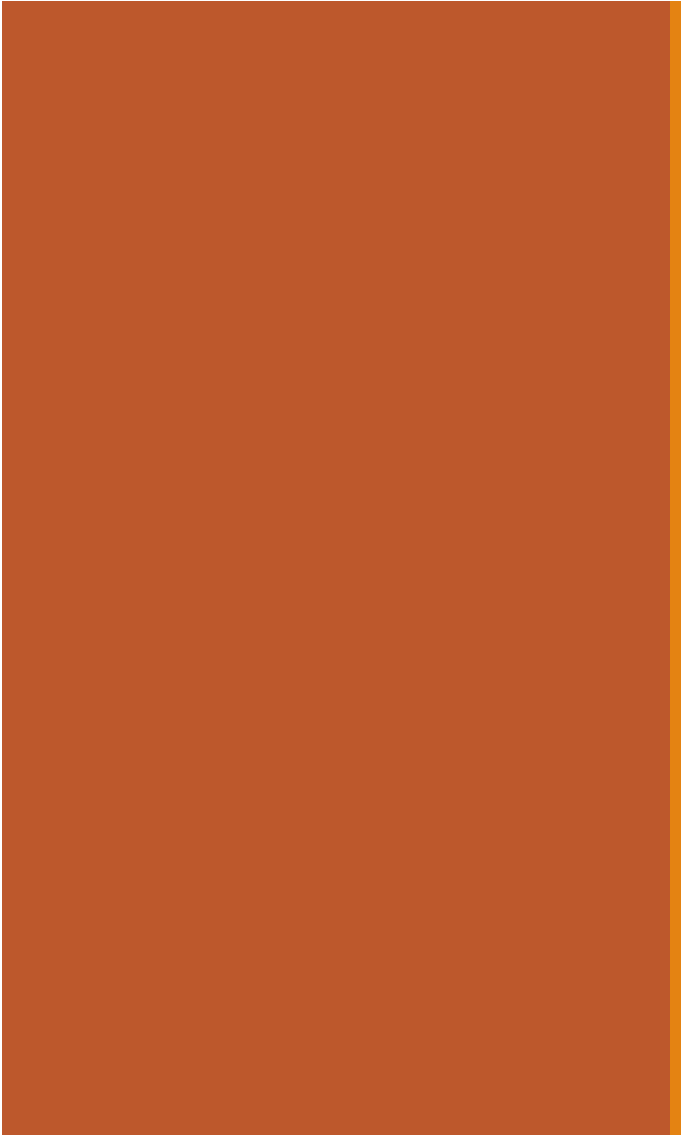
Training

Training data: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$
Learn parameters $\theta = (a, b)$

Two approaches:

- Analytical solution via mean squared error
- Iterative solution via MLE and gradient ascent

*drivable because
linear and one
dimensional*



Linear Regression: MSE

Mean Squared Error (MSE)

} also called Least Squares Error as well.

For most regressions, we generally choose a $g(X)$ that minimizes MSE: Y is the true value we observe, $\hat{Y}$ is a prediction of Y based on X , given by $g(X)$

$$\theta_{MSE} = \arg \min_{\theta} E[(Y - \hat{Y})^2] = \arg \min_{\theta} E[(Y - g(X))^2]$$

could have been $E[|\hat{Y} - Y|]$, but absolute value function is more difficult to manipulate because of behavior at 0.

- Y and $\hat{Y} = g(X)$ are both random variables
- Intuitively: Choose parameter θ that minimizes the expected squared deviation—what can be called an error—of your $\hat{Y}$ predictor from true Y

For linear regression, where $\hat{Y} = aX + b$ (so that $\theta = (a, b)$):

$$E[(Y - aX - b)^2]$$

} we choose a and b so that the expected value of this quantity is as low as possible.
 a_{MSE} and b_{MSE} are three things

Don't make me go nonlinear!

$$\theta_{MSE} = \arg \min_{\theta=(a,b)} E[(Y - aX - b)^2]$$

$$a_{MSE} = \rho(X, Y) \frac{\sigma_Y}{\sigma_X}, \quad b_{MSE} = \mu_Y - a_{MSE} \mu_X$$

(derivation included at the end of slides)

note that b_{MSE} is defined in terms of a_{MSE} , and we choose to not expand it, just to save space.

Can we compute these statistics for X and Y from our training data?

Training data: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$

Technically no, but **we can estimate them!**



Don't make me go nonlinear!

$$\theta_{MSE} = \arg \min_{\theta=(a,b)} E[(Y - aX - b)^2]$$

$$a_{MSE} = \rho(X, Y) \frac{\sigma_Y}{\sigma_X}, \quad b_{MSE} = \mu_Y - a_{MSE} \mu_X$$

(derivation included at the end of slides)

Can we compute these statistics for X and Y from our training data?

Training data: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$

Estimate parameters
based on observed
training data:

$$\hat{a}_{MSE} = \hat{\rho}(X, Y) \frac{S_Y}{S_X}, \quad \hat{b}_{MSE} = \bar{Y} - \hat{a}_{MSE} \bar{X}$$

sample
correlation

sample stdev

mean of all
 Y values in the
sample

mean of all
input values

$\hat{\rho}(X, Y)$:
Sample
correlation
([Wikipedia](#))

Summary: Linear Regression

Assume linear model (and $\mathbf{X}$ is 1-D):

$$\hat{Y} = g(\mathbf{X}) = aX + b$$

↙ predictor

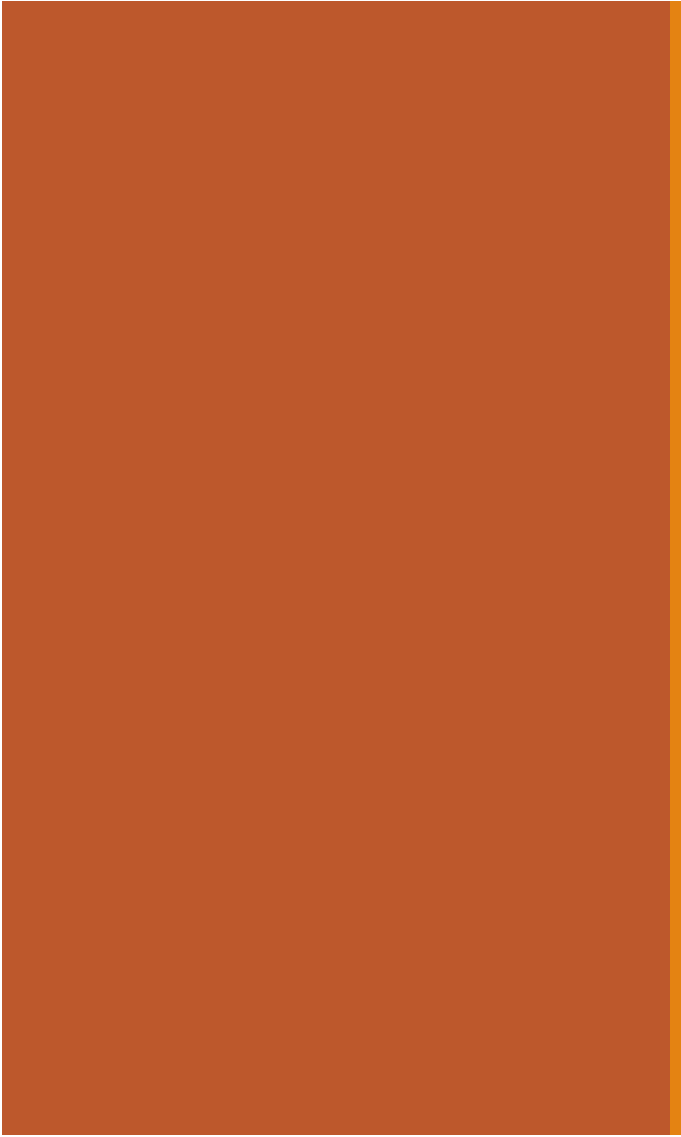
Training

Training data: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$
Learn parameters $\theta = (a, b)$

If we want to minimize the mean squared error of our prediction,

*general MSE theory
assumes we'll
compute these sample
statistics*

$$\hat{a}_{MSE} = \hat{\rho}(X, Y) \frac{S_Y}{S_X}, \quad \hat{b}_{MSE} = \bar{Y} - \hat{a}_{MSE} \bar{X}$$



Linear Regression: MLE

Linear Regression

Review

Assume linear model
(and $\mathbf{X}$ is 1-D, i.e., $\mathbf{X} = X$):

$$\hat{Y} = g(\mathbf{X}) = aX + b$$

Training

Learn parameters $\theta = (a, b)$

Training data: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$

We've seen which parameters—that is, what choices of a and b —minimize mean squared error: a_{MSE} and b_{MSE} , estimated by $\hat{a}_{MSE}$ and $\hat{b}_{MSE}$.

What if we want parameters that maximize the **likelihood of the training data**?

Note: Maximizing likelihood is typically an objective for classification models.

Likelihood, it's been a minute

Review

for example, $X_i \sim \text{Poi}(\theta)$

Consider a sample of n iid random variables $X_1, X_2, \dots, X_n$.

- X_i was drawn from some distribution with density function $f(X_i|\theta)$.
- Observed sample: $(X_1, X_2, \dots, X_n)$

Likelihood question:

How likely is the observed sample $(X_1, X_2, \dots, X_n)$ given parameter θ ?

Likelihood function, $L(\theta)$:

$$L(\theta) = f(X_1, X_2, \dots, X_n|\theta) = \prod_{i=1}^n f(X_i|\theta)$$

you've seen this
before for MLE
and MAP

This is just a product, since X_i are iid

Likelihood of the training data

Training data (n datapoints):

(shorthand)

- $(x^{(i)}, y^{(i)})$ drawn iid from a distribution $f(X = x^{(i)}, Y = y^{(i)} | \theta) = f(x^{(i)}, y^{(i)} | \theta)$
- $\hat{Y} = g(X)$, where g is a function on (X) and parameter θ

We can show that θ_{MLE} maximizes the **log conditional likelihood** function:

$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(x^{(i)}, y^{(i)} | \theta)$$

this is more difficult to
manipulate, so we work
with this instead



Linear Regression, MLE

1. Assume linear model for $\hat{Y}$
(and X is 1-D):

$$\hat{Y} = g(X) = aX + b$$

2. Define maximum likelihood estimator:

$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

⚠ Drama: We have a model for $\hat{Y}$, not Y

- Remember the MSE approach, where we minimize the squared **error** between $\hat{Y}$ and Y ?
- Here we **model this error** directly!

$$\begin{aligned} Y &= \hat{Y} + Z \\ &= aX + b + Z \end{aligned} \quad \begin{array}{l} \text{error/noise} \\ \text{(also random)} \end{array}$$

Comparison: MSE vs MLE

$$\hat{Y} = g(\mathbf{X}) = aX + b$$

Minimum Mean Squared Error

$$\theta_{MSE} = \arg \min_{\theta} E \left[(Y - g(X))^2 \right]$$

- Don't directly model Y (or any errors)
- Parameters are estimates of statistics from training data:

$$\begin{aligned}\hat{a}_{MSE} &= \hat{\rho}(X, Y) \frac{S_Y}{S_X} \\ \hat{b}_{MSE} &= \bar{Y} - \hat{a}_{MSE} \bar{X}\end{aligned}$$

Maximum Likelihood Estimation

$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

- Directly model error between predicted $\hat{Y}$ and Y as an RV Z

$$Y = \hat{Y} + Z = aX + b + Z$$

If we assume error $Z \sim \mathcal{N}(0, \sigma^2)$, then these two estimators are **equivalent**.

$$\theta_{MSE} = \theta_{MLE}!$$

Linear Regression, MLE (next steps)

1. Assume linear model (and $\mathbf{X}$ is 1-D):

$$\hat{Y} = g(\mathbf{X}) = aX + b$$

2. Define maximum likelihood estimator:

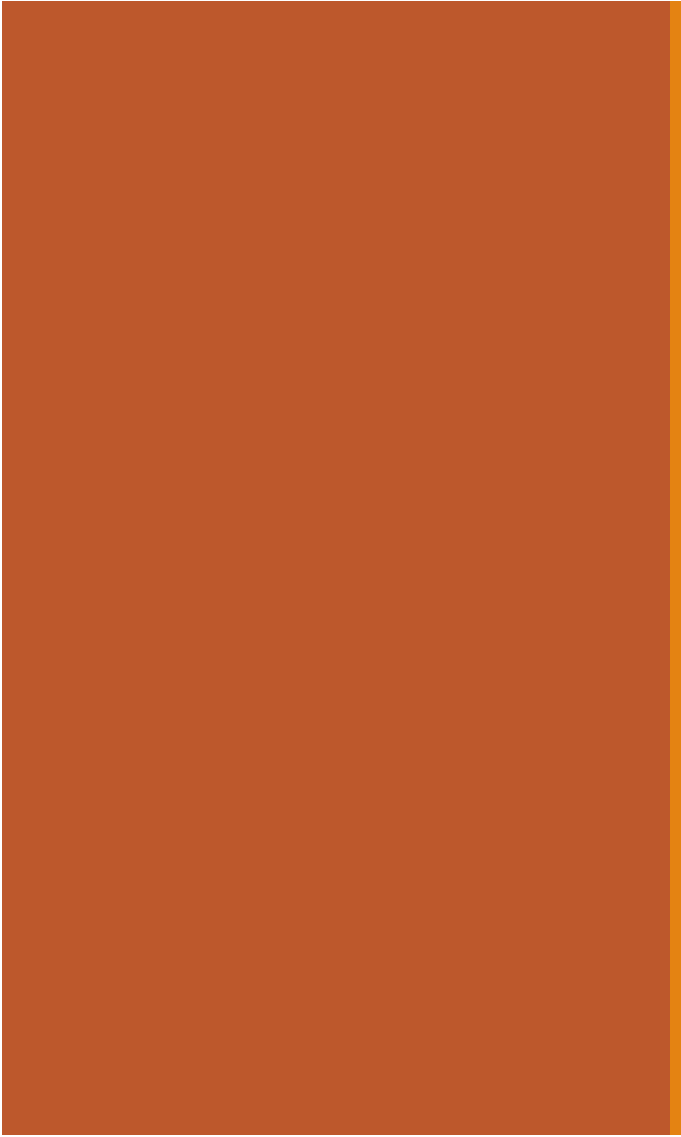
$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

3. Model error, Z :
and also model Y in terms of θ and Z .

$$Y = aX + b + Z, \text{ where } Z \sim \mathcal{N}(0, \sigma^2)$$

4. Pick $\theta = (a, b)$ that maximizes likelihood of training data

We won't find a solution analytically. Instead, we'll leverage **gradient ascent**, an **iterative optimization algorithm**.



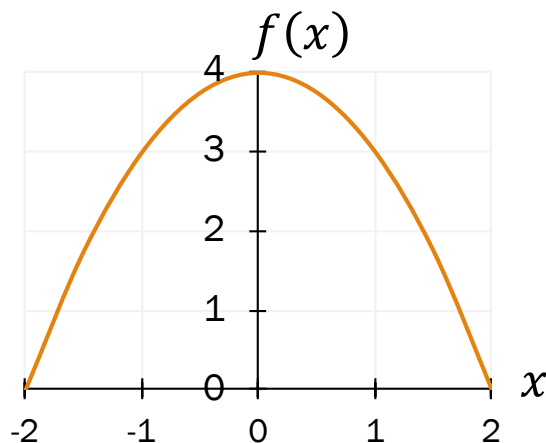
Gradient Ascent

Multiple ways to calculate argmax

Let $f(x) = -x^2 + 4$,
where $-2 < x < 2$.

What is $\arg \max_x \underbrace{f(x)}_{\text{objective function}}?$

A. Graph and guess

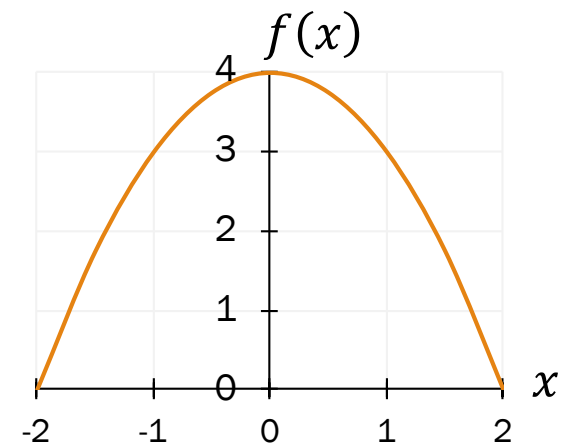


B. Differentiate,
set derivative to
0, and solve

$$\frac{df}{dx} = -2x = 0$$

$$x = 0$$

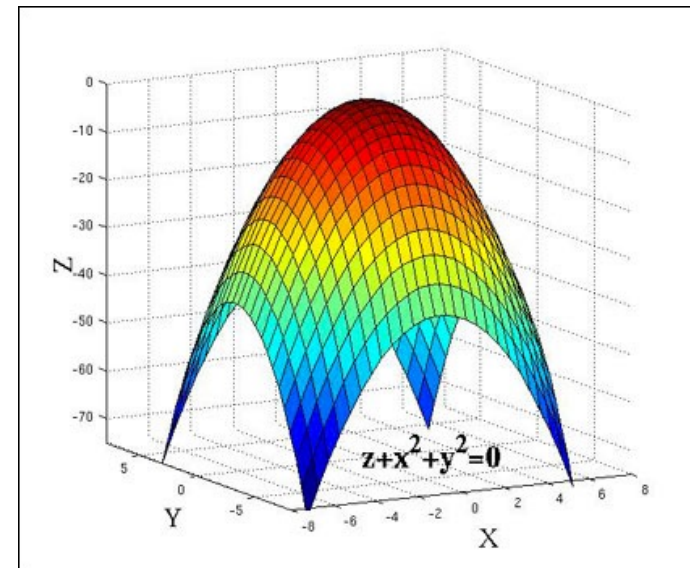
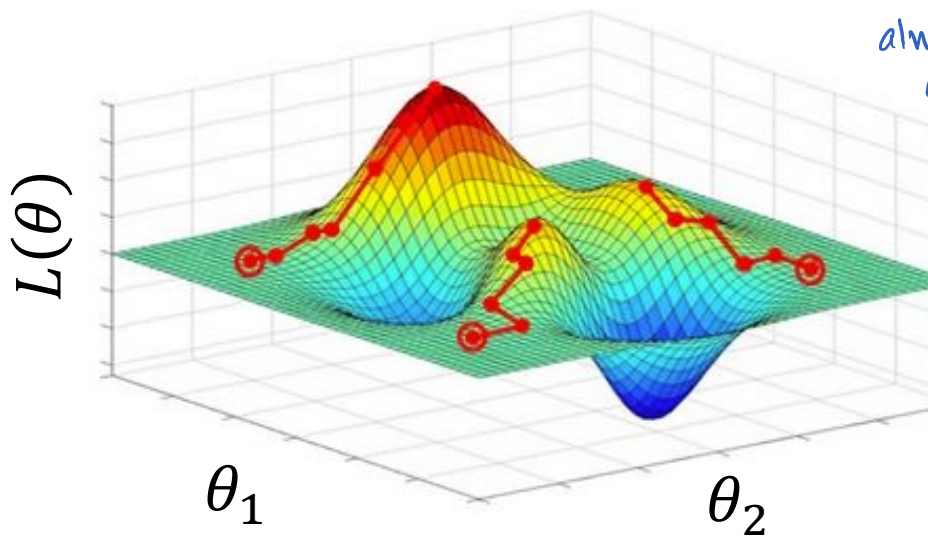
C. Gradient ascent: educated
guess & iteratively update



Gradient ascent

Walk uphill and you'll find a local maxima
(if your step is small enough).

*In CS109, $L(\theta)$
and $U(\theta)$ are
always
concave*

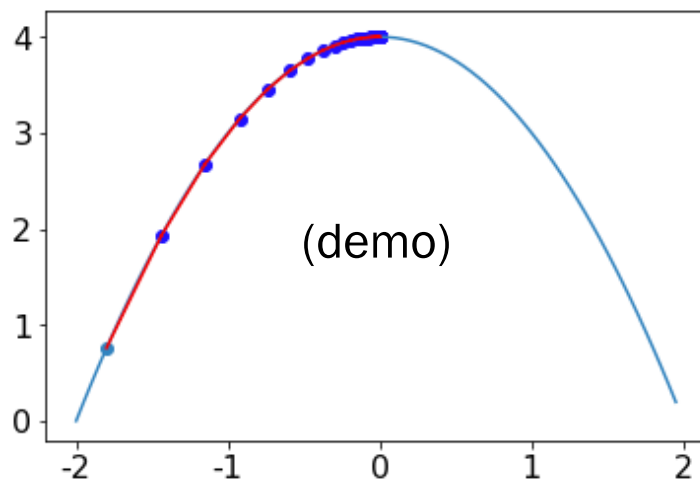


If your function is concave,
Local maxima = global maxima

Gradient ascent algorithm

Walk uphill and you'll find a local maxima
(if your step is small enough).

Let $f(x) = -x^2 + 4$,
where $-2 < x < 2$.



1. $\frac{df}{dx} = -2x$

pretend that gradient is so complicated that we can't just set equal to 0 and solve. Instead we discover solution numerically using

2. Gradient ascent algorithm:

initialize x

init to some reasonable guess if possible

repeat many times:

compute gradient

$x \leftarrow x + \eta * \text{gradient}$

we call this a learning rate

Linear Regression, MLE (so far)

Assume linear model
(and $\mathbf{X}$ is 1-D):

$$\hat{Y} = g(\mathbf{X}) = aX + b$$

Model Y as $\hat{Y} + Z$:

$$Y = aX + b + Z, \text{ where } Z \sim \mathcal{N}(0, \sigma^2)$$

Pick $\theta = (a, b)$ that maximizes
likelihood of training data

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} LL(\theta) \\ &= \arg \max_{\theta} \sum_{i=1}^n \log f(x^{(i)}, y^{(i)}, |\theta) \\ &= \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)\end{aligned}$$

(θ_{MLE} also maximizes
log conditional likelihood)

Computing the MLE with gradient ascent

General approach for finding θ_{MLE} , the MLE of θ :

1. Determine formula for $LL(\theta)$

log conditional likelihood

$$\sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

2. Differentiate $LL(\theta)$ w.r.t. (each) θ

$$\frac{\partial}{\partial \theta_j} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

$\theta_1 \quad \theta_2$
 $\downarrow \quad \downarrow$
 $\theta = (a, b)$

3. Solve resulting equations

(computer)
Gradient Ascent

1. Determine formula for log conditional likelihood

Model: $\theta = (a, b)$
 $Y = aX + b + Z$
 $Z \sim \mathcal{N}(0, \sigma^2)$

Optimization problem: $\arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$

Over the next few slides, we will show that our MLE linear regression θ_{MLE} reduces to

$$\arg \max_{\theta} \left[- \underbrace{\sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2}_{\text{objective function}} \right]$$

1. Determine formula for log conditional likelihood

Model: $\theta = (a, b)$

$$Y = aX + b + Z$$

$$Z \sim \mathcal{N}(0, \sigma^2)$$

Optimization
problem:

$$\arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

goal →

$$\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$

1. What is the conditional distribution, $Y|X, \theta$?
2. Substitute 1. into objective fn.
3. Use argmax properties to simplify objective fn.



1. Determine formula for log conditional likelihood

Model: $\theta = (a, b)$

$$Y = aX + b + Z$$

$$Z \sim \mathcal{N}(0, \sigma^2)$$

Optimization
problem:

$$\arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

$$Y | X=x, \theta=(a,b)$$

$$Y = aX + b + Z$$

1. What is the conditional distribution, $Y|X, \theta$?

$$Y|X, \theta \sim \mathcal{N}(aX + b, \sigma^2)$$

$$f(y^{(i)} | x^{(i)}, \theta) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(y^{(i)} - (ax^{(i)} + b))^2 / (2\sigma^2)}$$

2. Substitute 1. into objective fn.

$$\arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta) = \arg \max_{\theta} \sum_{i=1}^n \log \left[\frac{1}{\sqrt{2\pi}\sigma} e^{-(y^{(i)} - ax^{(i)} - b)^2 / (2\sigma^2)} \right]$$

$$\begin{aligned} \text{using} \\ \text{natural log} \end{aligned} \quad = \arg \max_{\theta} \left[\sum_{i=1}^n -\log \sqrt{2\pi}\sigma - \frac{1}{2\sigma^2} \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$

1. Determine formula for log conditional likelihood

Model: $\theta = (a, b)$

$$Y = aX + b + Z$$

$$Z \sim \mathcal{N}(0, \sigma^2)$$

Optimization
problem:

$$\arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

3. Use argmax properties
to simplify objective fn.

$$\arg \max_{\theta} \left[\sum_{i=1}^n \underbrace{-\log \sqrt{2\pi}\sigma}_{\text{doesn't depend on } \theta = (a,b)} - \frac{1}{2\sigma^2} \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right] \quad (\text{from previous slide})$$

$$= \arg \max_{\theta} \left[\underbrace{-\frac{1}{2\sigma^2} \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2}_{\text{invariant constant with regard to arg max}} \right]$$

$$= \arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$

Argmax refresher #1:

Invariant to additive constants

Argmax refresher #2:

Invariant to positive constant scalars

1. Determine formula for log conditional likelihood

Model: $\theta = (a, b)$

$Y = aX + b + Z$

$Z \sim \mathcal{N}(0, \sigma^2)$

Optimization
problem:

$$\arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

4. Celebrate!

$$\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$



Computing the MLE with gradient ascent

General approach for finding θ_{MLE} , the MLE of θ :

1. Determine formula for $LL(\theta)$

log conditional likelihood

$$\sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

$$h(\theta) = - \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2$$

2. Differentiate $LL(\theta)$ w.r.t. (each) θ

$$\frac{\partial}{\partial \theta_j} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

2-D gradient:

$$\left(\frac{\partial h(\theta)}{\partial a}, \frac{\partial h(\theta)}{\partial b} \right)$$

3. Solve resulting (simultaneous) equations

(computer)
Gradient Ascent

2. Compute gradient

Model: $\theta = (a, b)$
 $Y = aX + b + Z$
 $Z \sim \mathcal{N}(0, \sigma^2)$

Optimization
problem:

$$\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$

1. What is the derivative of the objective function w.r.t. a ?

$$\frac{\partial}{\partial a} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right] =$$

2. What is the derivative of the objective function wrt b ?



2. Compute gradient

Model: $\theta = (a, b)$
 $Y = aX + b + Z$
 $Z \sim \mathcal{N}(0, \sigma^2)$

Optimization
problem:

$$\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$

1. What is the derivative of the objective function w.r.t. a ?

$$\begin{aligned} \frac{\partial}{\partial a} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right] &= - \sum_{i=1}^n \frac{d}{da} (y^{(i)} - ax^{(i)} - b)^2 \\ &= -2 \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b) (-x^{(i)}) \\ &= 2 \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b) x^{(i)} \end{aligned}$$

Calculus refresher #1:

Derivative(sum) = sum(derivative)

Calculus refresher #2:

Chain rule 🌟🌟🌟

2. Compute gradient

Model: $\theta = (a, b)$

$$Y = aX + b + Z$$

$$Z \sim \mathcal{N}(0, \sigma^2)$$

Optimization
problem:

$$\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$

1. What is the derivative of the objective function wrt a ?

$$\sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$$

2. What is the derivative of the objective function wrt b ?

$$\sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

we could do this, but we're going to pretend it's impossible, as it often will be for more difficult objective functions.

analytical solution for a_{MLE}, b_{MLE} : Set to 0 and solve simultaneous equations

Next up: We will reach the same solution computationally with **gradient ascent**.

Computing the MLE with gradient ascent

General approach for finding θ_{MLE} , the MLE of θ :

1. Determine formula for $LL(\theta)$
log conditional likelihood

$$\sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

2. Differentiate $LL(\theta)$ wrt (each) θ

$$\frac{\partial}{\partial \theta_j} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

$$h(\theta) = - \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2$$

$$\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$$

$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

3. Solve resulting (simultaneous) equations

(computer)
Gradient Ascent

3. Gradient ascent with multiple parameters (if time)

Optimization problem:

$$\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$$
$$= \arg \max_{\theta} h(\theta)$$

Gradient:

$$\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$$
$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
initialize  $\theta$   
repeat many times:  
  compute gradient  
   $\theta \ += \ \eta \ * \ \text{gradient}$ 
```

How does this
work for
multiple
parameters?

3. Gradient ascent with multiple parameters

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$= \arg \max_{\theta} h(\theta)$

$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$

```
a, b = 0, 0 # initialize  $\theta$ 
repeat many times:
    gradient_a, gradient_b = 0, 0
    # TODO: fill in

    a +=  $\eta$  * gradient_a #  $\theta$  +=  $\eta$  * gradient
    b +=  $\eta$  * gradient_b
```

How do we
pseudocode the
gradients we
derived?

3. Gradient ascent with multiple parameters

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$= \arg \max_{\theta} h(\theta)$

$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$

```
a, b = 0, 0 # initialize  $\theta$   
repeat many times:
```

```
    gradient_a, gradient_b = 0, 0  
    for each training example (x, y):  
        diff = y - (a * x + b)  
        gradient_a += 2 * diff * x  
        gradient_b += 2 * diff
```

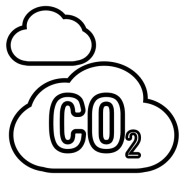
```
    a +=  $\eta$  * gradient_a #  $\theta$  +=  $\eta$  * gradient  
    b +=  $\eta$  * gradient_b
```

Finish computing
gradient before
updating any part
of θ .

([demo](#))

Global land-ocean temperature prediction

Training data: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$

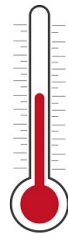


CO2 levels

Year 1	338.8
Year 2	340.0
...	
Year n	340.76

$X = (X_1)$

(assume one feature)



Output

0.26
0.32
$\vdots$
0.14

$Y \in \mathbb{R}$

Minimizing
Mean Square Error

Review

$$\theta_{MSE} = \arg \min_{\theta} E \left[(Y - g(X))^2 \right]$$

$$\hat{Y} = \hat{\rho}(X, Y) \frac{S_Y}{S_X} (X - \bar{X}) + \bar{Y}$$

$$a_{MSE} = 0.01452$$

$$b_{MSE} = 0.17511$$

3b. Interpret

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

$$= \arg \max_{\theta} h(\theta)$$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
a, b = 0, 0                # initialize  $\theta$ 
repeat many times:
    gradient a, gradient b = 0, 0
    for each training example (x, y):
        diff = y - (a * x + b)
        gradient_a += 2 * diff * x
        gradient_b += 2 * diff
    a +=  $\eta$  * gradient_a    #  $\theta$  +=  $\eta$  * gradient
    b +=  $\eta$  * gradient_b
```

Updates to a and b should include information from all n training datapoints

3b. Interpret

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

$$= \arg \max_{\theta} h(\theta)$$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
a, b = 0, 0          # initialize  $\theta$ 
repeat many times:
    gradient_a, gradient_b = 0, 0
    for each training example (x, y):
        diff = y - (a * x + b)
        gradient_a += 2 * diff * x
        gradient_b += 2 * diff
    a +=  $\eta$  * gradient_a    #  $\theta$  +=  $\eta$  * gradient
    b +=  $\eta$  * gradient_b
```

How do we interpret
the contribution of
the i-th training
datapoint?



3b. Interpret

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

$$= \arg \max_{\theta} h(\theta)$$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
a, b = 0, 0 # initialize  $\theta$   
repeat many times:
```

```
    gradient_a, gradient_b = 0, 0  
    for each training example (x, y):  
        diff = y - (a * x + b)  
        gradient_a += 2 * diff * x  
        gradient_b += 2 * diff
```

```
a +=  $\eta$  * gradient_a #  $\theta$  +=  $\eta$  * gradient  
b +=  $\eta$  * gradient_b
```

Prediction error!

$$y^{(i)} - \hat{y}^{(i)}$$

3b. Interpret

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

$$= \arg \max_{\theta} h(\theta)$$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
a, b = 0, 0                # initialize  $\theta$ 
repeat many times:
    gradient_a, gradient_b = 0, 0
    for each training example (x, y):
        prediction_error = y - (a * x + b)
        gradient_a += 2 * prediction_error * x
        gradient_b += 2 * prediction_error
    a +=  $\eta$  * gradient_a    #  $\theta$  +=  $\eta$  * gradient
    b +=  $\eta$  * gradient_b
```

3b. Interpret

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

$$= \arg \max_{\theta} h(\theta)$$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
a, b = 0, 0          # initialize  $\theta$   
repeat many times:
```

```
    gradient_a, gradient_b = 0, 0  
    for each training example (x, y):  
        prediction_error = y - (a * x + b)  
        gradient_a += 2 * prediction_error * x  
        gradient_b += 2 * prediction_error
```

```
    a +=  $\eta$  * gradient_a    #  $\theta$  +=  $\eta$  * gradient  
    b +=  $\eta$  * gradient_b
```

$\hat{Y} = aX + b$, so
update to a should
also scale by $x^{(i)}$

3b. Interpret

Optimization problem: $\arg \max_{\theta} \left[- \sum_{i=1}^n (y^{(i)} - ax^{(i)} - b)^2 \right]$

$$= \arg \max_{\theta} h(\theta)$$

Gradient: $\frac{\partial h(\theta)}{\partial a} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)(x^{(i)})$

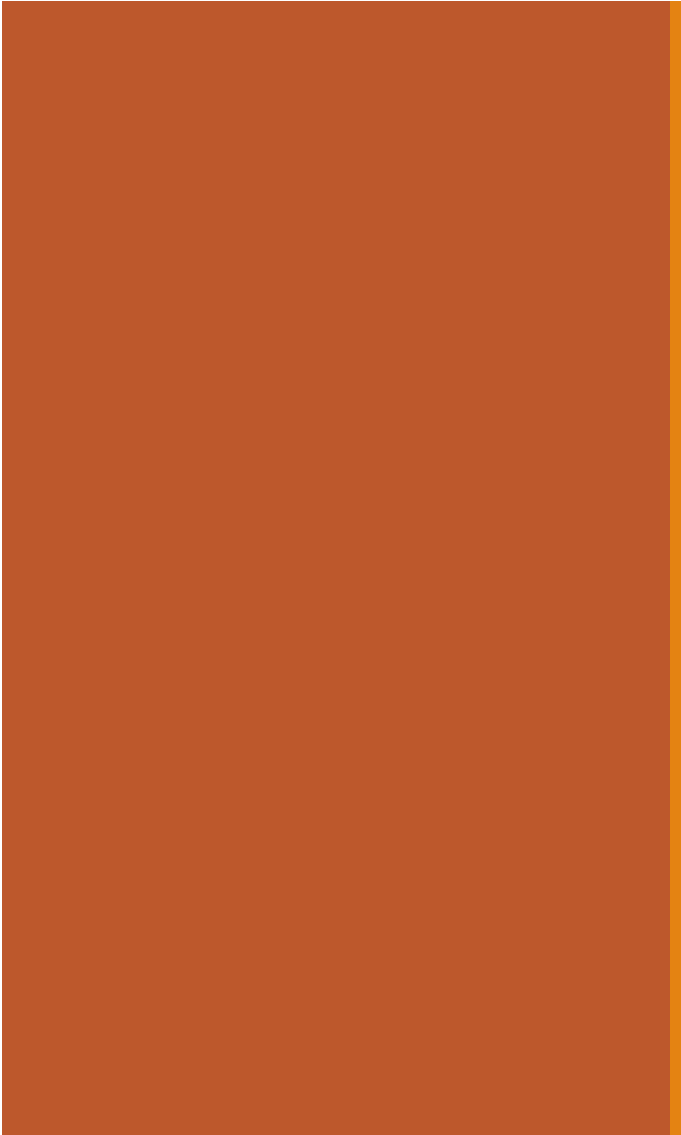
$$\frac{\partial h(\theta)}{\partial b} = \sum_{i=1}^n 2(y^{(i)} - ax^{(i)} - b)$$

```
a, b = 0, 0          # initialize  $\theta$   
repeat many times:
```

```
    gradient_a, gradient_b = 0, 0  
    for each training example (x, y):  
        prediction_error = y - (a * x + b)  
        gradient_a += 2 * prediction_error * x  
        gradient_b += 2 * prediction_error * 1
```

```
    a +=  $\eta$  * gradient_a    #  $\theta$  +=  $\eta$  * gradient  
    b +=  $\eta$  * gradient_b
```

$\hat{Y} = aX + b$, so
update to b just
scales by 1



Extra: Derivations

Don't make me get nonlinear!

$$\theta_{MSE} = \arg \min_{\theta=(a,b)} E[(Y - aX - b)^2]$$

1. Differentiate wrt (each) θ , set to 0

$$\begin{aligned} \frac{\partial}{\partial a} E[(Y - aX - b)^2] &= E \left[\frac{\partial}{\partial a} (Y - aX - b)^2 \right] && (E[\cdot] \text{ is a linear function w.r.t. } a) \\ &= E[-2(Y - aX - b)X] \\ &= -2E[XY] + 2aE[X^2] + 2bE[X] \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial b} E[(Y - aX - b)^2] &= E[-2(Y - aX - b)] \\ &= -2E[Y] + 2aE[X] + 2b \end{aligned}$$

2. Solve resulting simultaneous equations

$$a_{MSE} = \frac{E[XY] - E[X]E[Y]}{E[X^2] - (E[X])^2} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} = \rho(X, Y) \frac{\sigma_Y}{\sigma_X}$$

$$b_{MSE} = E[Y] - a_{MSE}E[X] = \mu_Y - \rho(X, Y) \frac{\sigma_Y}{\sigma_X} \mu_X$$

Log conditional likelihood, a derivation

$\hat{Y} = g(X)$, where $g(\cdot)$ is a function with parameter θ

Show that θ_{MLE} maximizes the **log conditional likelihood** function:

$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta)$$

Proof:

$$\begin{aligned} \theta_{MLE} &= \arg \max_{\theta} \prod_{i=1}^n f(x^{(i)}, y^{(i)} | \theta) &&= \arg \max_{\theta} \sum_{i=1}^n \log f(x^{(i)}, y^{(i)} | \theta) && (\theta_{MLE} \text{ also maximizes } LL(\theta)) \\ &= \arg \max_{\theta} \sum_{i=1}^n \log f(x^{(i)} | \theta) + \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta) && (\text{chain rule, log of product = sum of logs}) \\ &= \arg \max_{\theta} \sum_{i=1}^n \log f(x^{(i)}) + \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta) && (x^{(i)} \text{ indep. of } \theta) \\ &= \arg \max_{\theta} \sum_{i=1}^n \log f(y^{(i)} | x^{(i)}, \theta) && (f(x^{(i)}) \text{ constant w.r.t. } \theta) \end{aligned}$$